Editorial: Problem F - Rewrite The Stars

Setter: Hudson

Problem Overview

We are tasked with maintaining the **Maximum Weight Independent Set (MWIS)** of a dynamic forest under the following operations:

1. **Link(u, v):** Add an edge between two distinct trees.
2. **Cut(u, v):** Remove an existing edge.
3. **Update(u, w):** Update the weight of vertex u.
4. **Query(u):** Return the MWIS of the connected component containing u.

Difficulty Analysis

Rating: **4000+ (Legendary Grandmaster)**. This problem represents the pinnacle of dynamic tree data structures and is likely the hardest problem in the contest.

- Standard **Dynamic DP** (via Heavy-Light Decomposition) can handle weight updates but cannot handle structural changes (Link/Cut).
- Standard **Link-Cut Trees** (LCT) handle structural changes but are designed for path aggregates. They cannot easily maintain subtree aggregates (like MWIS) because the "dashed edges" in an LCT are not dynamically aggregated in a way that supports non-invertible DP transitions efficiently without complex auxiliary structures.

To solve this, one must implement a **Self-Adjusting Top Tree** (specifically the Compress-Rake Cluster variant). This data structure generalizes LCTs to handle subtree aggregates dynamically by treating "Rake" operations (merging a subtree onto a path) as first-class citizens alongside "Compress" operations (merging paths).

Solution Approach

The core idea is to decompose the tree into **Clusters**. A cluster corresponds to a connected subgraph with at most two **Boundary Nodes** interacting with the rest of the tree.

For MWIS, we define the state of a cluster (u, v) (where u, v are boundaries) using a 2×2 matrix M:

$$M[a][b] = \text{Max weight IS of cluster s.t. } u \text{ has state } a, v \text{ has state } b$$

State 0 $\implies$ Excluded, State 1 $\implies$ Included.

Top Tree Operations

The Top Tree maintains the structure using two merge types:

1. **Compress (Path Merge):** Merges two clusters (u, k) and (k, v) into (u, v). The transition resembles matrix multiplication in the $(\max, +)$ semiring.

$$M_{new}[a][b] = \max(L[a][0] + R[0][b], L[a][1] + R[1][b] - W_k)$$

   (Note: W_k is subtracted to prevent double-counting the shared boundary node k.)

2. **Rake (Subtree Merge):** Merges a cluster (k, k) (effectively a pending subtree) onto a path cluster at node k. This updates the internal weight/state of node k within the path cluster to reflect the optimal choice from its subtree.

Implementation Complexity

This solution requires implementing a full Top Tree with:

- Lazy propagation (for path reversals).
- Ternary Splay operations (handling Left Compress, Right Compress, and Middle Rake children).
- 'Access', 'Link', and 'Cut' operations that explicitly manage Rake edges.

A correct implementation typically exceeds 500 lines of C++. Due to the complexity, the time limit is set generously to 8.0s.